Name: Abdulaziz Mohammed Alkhlawi

ID: 2240006095

Lab 7 — Large Language Models (LLMs)

Completed lab solution. Original instructions: NLP_Lab7.

Run the cells from top to bottom. Results below were generated by executing this notebook. The `dataset` folder contains the original data needed by this lab. Missing Python packages are installed by the setup cell; pretrained model downloads need internet on their first run. Random seeds are fixed where supported.

```
# Install only missing dependencies into the active Jupyter Python environment.
import importlib.util, subprocess, sys, os, random
from pathlib import Path
packages = {'numpy': 'numpy', 'pandas': 'pandas', 'matplotlib': 'matplotlib', 'sklearn': 'scikit-learn',
'nltk': 'nltk', 'torch': 'torch', 'transformers': 'transformers==4.57.6', 'sentencepiece': 'sentencepiece'}
missing = [package for module, package in packages.items()
           if importlib.util.find_spec(module) is None]
if missing:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing])
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['USE_TF'] = '0'  # Transformers examples use PyTorch.
os.environ['USE_FLAX'] = '0'
import numpy as np
import pandas as pd
from IPython.display import display, Markdown, HTML
random.seed(42)
np.random.seed(42)
pd.set_option('display.max_colwidth', 90)
pd.set_option('display.max_columns', 10)
DATA = Path('dataset')
assert DATA.exists() or 7 in (7, 8, 9), 'Open this notebook from its Lab folder.'
```

The Three Main Transformer Model Families

Family	Example Models	Main Use
Encoder-only	BERT, RoBERTa	Classification, sentiment analysis, NER
Encoder–decoder	T5, BART	Translation, summarization, question answering
Decoder-only	GPT family	Text generation, conversation, completion

1. Setup

```
import torch
torch.manual_seed(42)
torch.set_num_threads(2)
print('PyTorch:',torch.__version__)
```

```
PyTorch: 2.15.0+cpu
```

```
from transformers import pipeline, set_seed

print("Libraries loaded successfully!")
```

```
C:\Users\Azoz\Documents\Codex\2026-11-05\https-github-com-mohammedmosuily-nlp-lab1-3\work\venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
```

```
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Libraries loaded successfully!
```

Part A — Decoder-Only Model

Decoder-only example: GPT-style Text Generation

A decoder-only model predicts the next token based on previous tokens.

We will use **DistilGPT-2**, a small GPT-style model.

```
generator = pipeline("text-generation", model="distilgpt2", device=-1)

prompt = "Natural language processing is"

output = generator(prompt, max_new_tokens=30, do_sample=False, pad_token_id=generator.tokenizer.eos_token_id)

print(output[0]["generated_text"])
```

```
Device set to use cpu
```

```
The following generation flags are not valid and may be ignored: ['temperature']. Set
`TRANSFORMERS_VERBOSITY=info` for more details.
```

```
Natural language processing is a common feature of the language processing process.
```

Part B — Encoder–Decoder Model

Encoder–decoder example: Instruction / Text-to-Text Generation

Encoder–decoder models contain two main parts:

1. The **encoder** reads the input.
2. The **decoder** produces the output.

We will use **FLAN-T5-small**.

```python
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

prompt = "Translate to French: I love natural language processing."

# Convert text into tokens
output = model.generate(**tokenizer(prompt, return_tensors="pt"),max_new_tokens=50)

# Convert tokens back to text
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
Je s'agissant de la technologie de linguage naturelle.
```

Part C — Encoder-Only Model

Encoder-only example: Sentiment Classification

Encoder-only models read the full input and create a representation of it.

They are very useful when the task is to **understand or classify text**.

Here we use a DistilBERT sentiment model.

```python
classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

text = "This NLP lab is easy and interesting."

result = classifier(text)
print(result)
```

```
Device set to use cpu
```

```
[{'label': 'POSITIVE', 'score': 0.999778687953949}]
```

Tasks

Complete the following tasks.

Task 1 — Model Families

For each task below, choose the most suitable model family:

1. Sentiment classification
2. Machine translation
3. Open-ended text generation
4. Named Entity Recognition
5. Text summarization

```python
families = {'Sentiment classification':'Encoder-only', 'Machine translation':'Encoder-decoder',
            'Open-ended text generation':'Decoder-only', 'Named Entity Recognition':'Encoder-only',
            'Text summarization':'Encoder-decoder'}
display(pd.DataFrame(families.items(),columns=['Task','Suitable family']))
```

	Task	Suitable family
0	Sentiment classification	Encoder-only
1	Machine translation	Encoder-decoder
2	Open-ended text generation	Decoder-only
3	Named Entity Recognition	Encoder-only
4	Text summarization	Encoder-decoder

Task 2 — Encoder-only Model

Use the sentiment classifier on these three sentences:

```
The movie was excellent.
The lecture was difficult to understand.
I enjoyed learning about transformers.
```

Print the predicted label and score for each sentence.

```
sentences=['The movie was excellent.','The lecture was difficult to understand.',
           'I enjoyed learning about transformers.']
results=classifier(sentences)
display(pd.DataFrame([{'Sentence':s,**r} for s,r in zip(sentences,results)]))
```

	Sentence	label	score
0	The movie was excellent.	POSITIVE	0.999861
1	The lecture was difficult to understand.	NEGATIVE	0.999665
2	I enjoyed learning about transformers.	POSITIVE	0.999746

Task 3 — Encoder–Decoder Model

Use the FLAN-T5 model to perform the following:

- Summarize a short paragraph.

```
paragraph = ('The university library has extended its opening hours during exam week. '
             'It now opens at 7 a.m. and closes at midnight. Students can use the study rooms and borrow
laptops. '
             'The extended schedule will end on Friday.')
summary_input = 'Summarize the following paragraph in one sentence: ' + paragraph
summary_ids = model.generate(**tokenizer(summary_input,return_tensors='pt'),
                             max_new_tokens=60,num_beams=4,do_sample=False)
print('Paragraph:',paragraph)
print('Summary:',tokenizer.decode(summary_ids[0],skip_special_tokens=True))
```

```
Paragraph: The university library has extended its opening hours during exam week. It now opens at 7 a.m. and
closes at midnight. Students can use the study rooms and borrow laptops. The extended schedule will end on
Friday.
Summary: The university library has extended its opening hours during exam week. It is now open at 7 a.m. and
closes at midnight.
```

Task 5 — Short Reflection

Answer in 3–5 sentences:

Why is a decoder-only model suitable for a chatbot such as ChatGPT?

Task 5 answer

A decoder-only model generates a response one token at a time using the preceding conversation as context. Causal attention prevents access to future tokens and matches this next-token prediction objective. It can continue a dialogue in a flexible way rather than requiring a fixed set of response classes. Instruction tuning and feedback help the pretrained model follow requests and produce useful conversational responses.

Optional Challenge

Try generating text several times with the same prompt.

Remove:

```
set_seed(42)
```

and run the sampling cell multiple times.

Question

Why do you sometimes get different outputs even when the prompt is the same?

Hint: Sampling introduces randomness when choosing the next token.

```
import re
prompt='Natural language processing is'
for trial in range(3):
    sample=generator(prompt,max_new_tokens=25,do_sample=True,top_p=0.9,temperature=0.8,
                     pad_token_id=generator.tokenizer.eos_token_id)
    print(f'Sample {trial+1}:',re.sub(r'\s+',' ',sample[0]['generated_text']).strip())
```

```
Sample 1: Natural language processing is a complex concept and has been studied extensively. The use of language
processing techniques in this area is particularly relevant in our study of


Sample 2: Natural language processing is not available in this language.


Sample 3: Natural language processing is a great way to help beginners and teachers in the area. This language
processing has a high level of knowledge, skill and practice
```

Optional challenge answer

Sampling chooses among next-token possibilities at random, so successive calls can follow different paths. A fixed seed makes the sequence of draws reproducible; resetting the same seed before each identical call can reproduce the same output. Greedy decoding (`do_sample=False`) instead chooses the highest-probability token at each step. The lab's source skips Task 4; all tasks that are actually listed are completed here. Classifier scores are model confidence values, not guarantees of correctness.